10c. Introduction to SIMD

Martin Alfaro
PhD in Economics

INTRODUCTION

Single Instruction, Multiple Data (SIMD) is an optimization technique widely embraced in modern CPU architectures. At its core, SIMD allows a single CPU instruction to process multiple data points concurrently, rather than sequentially processing them one by one. This parallel approach can yield substantial performance gains, especially for workloads involving simple identical calculations repeated across multiple data elements. [1]

SIMD is a form of parallelism within a single CPU core. By operating on vectors rather than individual elements, SIMD instructions can execute the same operation on multiple data points simultaneously. This is why the process of applying SIMD is often referred to as **vectorization**.

To illustrate, consider a computation involving four separate addition operations. Without SIMD, the computer would need to execute four distinct instructions, one for each addition. Instead, SIMD makes it possible to bundle the four additions into a single instruction, allowing the CPU to process them all at once. In an ideal scenario, the time required to complete four additions with SIMD would be the same as completing one addition without it.

Throughout the different sections, we'll cover two approaches for implementing SIMD instructions.

- Julia's native capabilities.
- The package `LoopVectorization`.

This section will exclusively concentrate on the built-in tools for applying SIMD. In particular, we'll explore the conditions that trigger automatic vectorization. The next section we'll introduce the `@simd` macro, which lets the user manually suggest SIMD in for-loops. The discussion of `LoopVectorization` will be saved for later sections. This package implements more aggressive optimizations, but can also introduce bugs if misused.

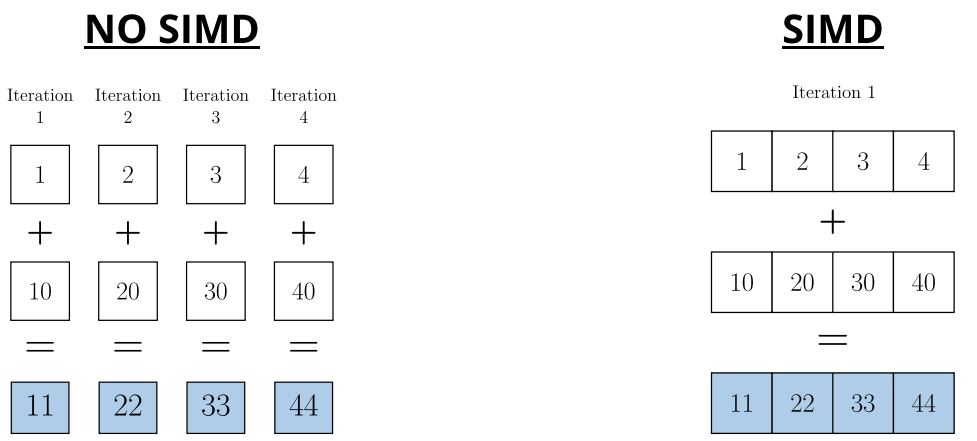
WHAT IS SIMD?

SIMD is a form of data-level parallelism within a single processor core, where the same operation is applied to multiple data elements at once. It's particularly effective for basic arithmetic operations, such as addition and multiplication. Given the nature of these operations, it's unsurprising that one of SIMD's primary applications is in linear algebra, where operations like vector addition or matrix multiplication involve performing identical arithmetic steps across many elements.

At the heart of SIMD lies the process of vectorization, where data is split into sub-vectors that can be processed as single units. To facilitate this operation, modern processors usually include dedicated SIMD registers to support this execution model. In particular, contemporary processors typically feature 256-bit registers for vectorized operations, which are wide enough to hold four `Float64` or `Int64` values at once.

To illustrate the workings of SIMD, consider adding two vectors, `x = [1, 2, 3, 4]` and `y = [10, 20, 30, 40]`. In traditional scalar execution, the processor would perform the operation `x + y` in four separate steps, one for each pair of numbers: `1+10`, `2+20`, `3+30`, `4+40`. Each addition requires its own instruction, so that the final output `[11, 22, 33, 44]` would take four sequential steps.

SIMD changes this dynamic entirely. Instead of issuing four independent additions, the processor loads the four elements of `x` and the four elements of `y` into a 256-bit SIMD register. Once the data is packed, a single vector-addition instruction triggers all four additions simultaneously. Internally, the hardware views the register as a collection of lanes, with each lane holding one element of the vector. The instruction applies the same operation to every lane in parallel, collapsing what would've been four scalar instructions into one. This ability to collapse multiple scalar operations into one instruction is what gives SIMD its substantial performance advantage in data-parallel workloads.



For larger vectors that exceed the width of a single register, the process remains fundamentally the same. The only difference is that processor first partitions the input into consecutive sub-vectors that fit the register size. Then, it performs the vectorized operation on each sub-vector, continuing until all elements have been processed.

BROADCASTING AND FOR-LOOPS

There are **two categories of operations that can potentially benefit from SIMD instructions: for-loops and broadcasting**. In fact, we could've explicitly mentioned only the former, since broadcasting is essentially syntactic sugar for for-loops.

In its built-in form, the decision to apply SIMD instructions is nonetheless made entirely by the compiler. Its decision relies on a set of heuristics to determine when vectorization will pay off.

For broadcasting in particular, the compiler implements SIMD automatically, without any user intervention. Instead, the automatic application of SIMD in for-loops is only restricted to a few simple cases, requiring the user to explicitly indicate that SIMD should be considered.

This is why the upcoming sections will identify conditions under which SIMD instructions should be suggested to the compiler. If these conditions aren't met, SIMD will substantially reduce its effectiveness or directly become infeasible. For such cases, we'll also provide guidance on how to structure computations so that SIMD remains efficient, despite not being well-suited for its application.

SIMD IN BROADCASTING

As we indicated, SIMD is automatically applied in broadcasting. To devote our full attention in subsequent sections to for-loops, we conclude this section by demonstrating this behavior in broadcasting.

The following example compares the same computation implemented via a for-loop and via broadcasting. While broadcasting automatically takes advantage of SIMD, the for-loop version doesn't.

FOR-LOOP (NO SIMD)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2 / x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  788.201 μs (3 allocations: 7.629 MiB)
```

BROADCASTING (SIMD)

```
x      = rand(1_000_000)

foo(x) = 2 ./ x
```

```
julia> @btime foo($x)
  411.572 μs (3 allocations: 7.629 MiB)
```

FOOTNOTES

[1.] SIMD isn't exclusive to CPUs. GPUs also take advantage of it. The architecture of GPUs is a natural fit for SIMD, as they were designed for parallel processing of certain simple identical operations.